

iHelp — Testing Specification

1. Testing Philosophy — what "it works" means here

iHelp's central claim is not "the buttons work" — it is that **the database enforces every rule in the product spec's §9.2 permission matrix, even against callers that skip the UI entirely**. The test suite is therefore weighted accordingly (assignment: "test the central processes, not every line"):

1. **The highest-value tests talk to a real PostgreSQL** (local Supabase) with real JWTs, attempting both the allowed action *and its forbidden twin*. A permission test that only checks the happy path proves nothing; every RLS test here asserts *denial* too — spec §13.2 makes exactly this a success criterion.
2. **Pure logic gets unit tests** — cheap, fast, no environment.
3. **UI gets a thin layer**: component render tests for shared vocabulary and one end-to-end pass over the core marketplace loop, because the loop working in a browser is the product working.

What is deliberately *not* tested: styling, static pages' content, Next.js framework behavior, and Supabase's own auth internals — none of these are our logic.

2. Test Layers, Tools, and Where They Run

Layer	Tool	Environment	Files
Unit — pure logic	Vitest	Node, no DB	<code>lib/geo.test.ts</code> , <code>lib/validation/*.test.ts</code>
Component — shared UI	Vitest + React Testing Library (jsdom)	No DB	<code>components/ui.test.tsx</code>
Integration — RLS / RPC / state machine	Vitest + <code>@supabase/supabase-js</code>	Local Supabase (<code>npx supabase start</code>)	<code>tests/integration/*.test.ts</code>
End-to-end — core loop	Playwright	Local Supabase + <code>next dev</code>	<code>e2e/core-flow.spec.ts</code>

Integration and E2E tests **skip automatically** when no local stack is detected (`SUPABASE_URL` absent), so `npm test` always passes in a bare checkout while the full suite runs where it matters.

The suite totals **64 Vitest tests** across 5 files — 33 integration (lifecycle 11 + permissions 22), geo 6, validation schemas 15, component UI 10 — plus 1 Playwright E2E spec, for **65 automated tests** in all.

3. What Is Tested and Why (assignment stage-6 checklist)

3.1 Central features & business processes (integration + E2E)

The full request lifecycle, exercised through real clients as real users:

#	Scenario	Proves
F1	Verified user creates a request via <code>create_request_with_photos</code> (with an uploaded photo) → status <code>open</code> , photos attached	The only write path into <code>help_requests</code> works and enforces its invariants
F1b	Verified user creates a request with no photos → status <code>open</code> , zero photos attached	Photos are optional (0–5); the write path accepts a photo-less request
F2	Second verified user submits an offer → request flips to <code>has_offers</code> (trigger T2)	Offer flow + automatic status sync
F3	Owner calls <code>assign_offer</code> → request <code>assigned</code> , chosen offer <code>selected</code> , competitor offer <code>closed</code>	The atomic pivotal moment, including competitor closure
F4	Both sides call <code>confirm_completion</code> → flags set per caller; status flips to <code>completed</code> only after the second call	Dual-sided completion semantics
F5	Owner calls <code>submit_rating</code> → rating row exists, request <code>rated</code> , helper aggregate visible via <code>helper_ratings</code>	Rating + atomic terminal transition
F6	Owner calls <code>cancel_request</code> mid-flow → request <code>cancelled</code> , live offers <code>closed</code>	The terminal escape hatch
F7	Withdraw last active offer → request returns to <code>open</code>	The <code>has_offers</code> ↔ <code>open</code> cycle
F8	<code>after_job</code> offer: no price up front; after dual completion the selected helper calls <code>set_final_price</code> (a stranger gets <code>not_found</code> ; <code>mark_paid</code> before pricing and a second <code>set_final_price</code> both get <code>invalid_state</code>)	Deferred pricing — the agreed amount is set once, by the selected helper only, post-completion
F9	<code>get_helper_stats</code> returns correct aggregates (completed jobs, rating count/avg, star distribution, per-category counts) for a helper with a completed + rated job; the <code>helper_ratings</code> view exposes <code>category</code> but selecting <code>rater_id / request_id</code> fails (columns absent)	Reputation aggregates are computed server-side and stay anonymized — category context without rater/request linkage
E2E	The F1–F5 chain driven through the real UI by two browser sessions	The product works as users experience it

3.2 Permission tests — different users attempting forbidden actions (integration)

One test per §9.2 matrix row that could be abused, always asserting **denial** (error or zero-row result — the silent-denial pattern):

#	Forbidden action attempted	Expected
P1	Unverified user calls <code>create_request_with_photos</code>	<code>forbidden</code>
P2	Unverified user inserts an offer directly	RLS insert denial
P3	Helper offers on their own request	RLS insert denial
P4	Offer inserted born <code>selected</code>	RLS insert denial (status pin)
P5	Non-owner reads someone else's offer (sealed bids)	zero rows
P6	Stranger reads an <code>assigned</code> request they're not party to	zero rows
P7	Non-owner calls <code>assign_offer</code> / <code>cancel_request</code>	<code>not_found</code> (existence-safe)
P8	Helper calls <code>submit_rating</code> ; stranger calls <code>confirm_completion</code>	<code>not_found</code>
P9	Owner PATCHes <code>status</code> / <code>is_paid</code> directly (crafted update)	guard-trigger <code>forbidden</code> / zero rows
P10	User PATCHes own <code>is_identity_verified</code> / <code>is_admin</code>	guard-trigger <code>forbidden</code>
P11	User reads another user's <code>profiles_private</code> row (phone/home coords)	zero rows
P12	Non-party calls <code>get_counterpart_contact</code>	<code>not_found</code> ; parties succeed only from <code>assigned</code>
P13	Non-admin calls <code>review_application</code> / <code>set_request_hidden</code> / <code>revoke_verification</code>	<code>forbidden</code>
P14	User inserts an application with <code>status='approved'</code> (forged audit row)	RLS insert denial
P15	Anonymous (no JWT) reads any table / <code>helper_ratings</code> view	denial / zero rows

3.3 Database tests (integration)

Constraints and triggers as the last line of defense:

#	Case	Expected
D1	Second <i>active</i> offer by same helper on same request	unique-index violation
D2	Second application of same kind while one is pending	partial-unique violation
D3	Offers carrying each pricing stance on the same request — pricing belongs to the offer, so a request accepts any mix	all three stances (<code>fixed</code> / <code>volunteer</code> / <code>after_job</code>) accepted
D4	Second rating for the same request	PK violation
D5	Professional application without a document	CHECK violation
D6	Signup trigger creates <code>profiles</code> + <code>profiles_private</code> rows	rows exist
D7	<code>helper_ratings</code> view exposes stars/note but no <code>rater_id</code> / <code>request_id</code>	column absence
D8	Identity revocation clears both flags and revokes pending professional applications	flags false, applications <code>revoked</code>
D9	<code>fixed</code> offer without a price / <code>volunteer</code> offer with a price	<code>price_matches_mode</code> CHECK violation
D10	Offer inserted with a pre-set <code>final_price</code>	CHECK violation — the insert pin keeps <code>final_price</code> null so the amount is only ever set via the guarded <code>set_final_price</code> RPC

3.4 Invalid-input tests (unit, zod schemas)

Every schema rejects what the DB would reject — same bounds, friendlier message: title/description/message length bounds, bad phone formats, stars outside 1–5, offer-price bounds, photo count (accepts 0, rejects 6 — photos are optional, max 5), **absent location (the (0,0) "Null Island" case)**, absent optional fields arriving as `null` (the FormData null-coercion case).

3.5 Edge cases (integration)

#	Case	Expected
X1	<code>assign_offer</code> on a just-withdrawn offer	<code>offer_not_active</code> , transaction rolled back — request still <code>has_offers</code>
X2	<code>confirm_completion</code> called twice by the same side	idempotent — still waiting for the other side
X3	<code>mark_paid</code> twice, or (F8) before an agreed amount exists (unpriced <code>after_job</code> offer)	<code>invalid_state</code>
X4	Approving a professional application after identity was revoked	<code>invalid_state</code>
X5	Rejection without a note	<code>note_required</code>
X6	When photos are supplied, paths pointing at another user's folder / nonexistent objects	<code>forbidden</code> / <code>photo_not_uploaded</code>

3.6 Basic UI tests (component + E2E)

Component: `Stars` (aggregate rendering + empty state), `StatusChip` (Hebrew labels per status), `Badge` (verified/professional combinations). The offer-price display (`OfferPriceChip`) is exercised by the E2E flow, which asserts the helper-quoted `₪120` renders on the request page. E2E also asserts the Hebrew RTL shell renders, forms submit, and the lifecycle panels appear for the right role at the right state.

4. Test Data Strategy

Integration tests are **self-contained**: each run creates its own throwaway users via the local stack's admin API (allowed: local tooling), promotes one to admin by direct SQL, and builds its fixtures through the same RPCs users would call. No dependence on `scripts/seed.ts` (which serves the demo, not the suite). E2E uses the seed script's known accounts.

5. Running

```
npm test           # unit + component (+ integration when stack is up)
npx supabase start # local stack for integration/E2E
npm run test:e2e   # Playwright core-flow spec (starts next dev itself)
```

6. Success Criteria for the Suite

The suite fulfills spec §13.2 ("every permission ... demonstrated by tests that attempt forbidden actions and observe denial") when every §3.2 row runs against the real database and fails **closed** — and the F-chain proves the happy path end to end.